ReAct 用於單元測試生成：Thought 真的有幫助嗎？

Mid-term Study Report —Software Testing, 2026 Fall

- 學號 / 姓名：31551039 / 吳堃瑋
- 主題類別：AI Agents for Software Testing（論文復現與延伸）
- 研究對象論文：Yao et al., ReAct: Synergizing Reasoning and Acting in Language Models, ICLR 2023
- 程式碼與實驗紀錄：https://github.com/ruserxd/st2026-midterm-aiagent（`ai-agent/`、`subject/`、`evaluation/`、`runs/`）

摘要

本報告把 ReAct 論文的核心消融實驗「ReAct vs Act」搬到**單元測試生成**。我在一個約 160 行的購物車計價模組中植入 4 個 bug，讓本機執行的 `gpt-oss:20b` agent 透過工具讀規格、寫測試、跑測試，再用**獨立的評分程式**判斷測試抓到幾個 bug、有幾個誤報、有沒有 assertion echo。主實驗有 5 個條件，每個條件跑 100 次（共 500 次）；延伸實驗另有 4 個條件（共 300 次）。

主要發現：

1. **few-shot 範例的影響遠大於 Thought**：抓到的 bug +0.83 個、誤報 −55%（$p < 0.001$）。
2. **要求 agent 寫 Thought 沒有幫助**：在 4 組比較中，沒有一組讓 agent 顯著抓到更多 bug；有 2 組讓 agent 顯著**少讀**規格，成本也一律增加。
3. 論文「ReAct 優於 Act」的主張，在本設定（小型推理模型、原生工具呼叫、測試生成）下**沒有重現**。
4. 延伸實驗顯示，真正有用的是**模型自己的推理**：把 gpt-oss 的內部思考留在上下文中，比要求寫 Thought 多抓 0.66 個 bug；把推理強度從 low 調到 high，多跳 bug 的抓到率從 4% 提高到 66%，代價是耗時約 10 倍。

---

1. Introduction

1.1 論文基本資訊

項目	內容
標題	ReAct: Synergizing Reasoning and Acting in Language Models
作者	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao
單位	Princeton University、Google Research（Brain team）
發表	ICLR 2023（arXiv:2210.03629）
實驗任務	HotpotQA（多跳問答）、FEVER（事實查核）、ALFWorld（文字遊戲）、WebShop（網購）

項目	內容
使用模型	PaLM-540B（prompting）；PaLM-8B/62B（finetuning）

1.2 設計動機與背景知識

在 ReAct 之前，LLM 的「推理」和「行動」大多分開研究：

- **只推理（Chain-of-Thought, CoT）[2]**：模型一步步寫出推理，但只依賴自己的內部知識，容易產生**幻覺**，錯誤也會沿著推理鏈傳下去。
- **只行動（Act）**：模型直接輸出動作（例如搜尋 Wikipedia），能取得外部資訊，但不會規劃、不會追蹤進度，也不會判斷觀察結果代表什麼。

ReAct 的想法是讓兩者**交錯進行**：推理用來規劃、追蹤進度、處理例外；行動用來取得外部資訊，修正推理。

1.3 架構

論文把 agent 的動作空間從 A 擴充為 $\hat{A} = A \cup L$，L 是語言空間。「Thought」是一種**不影響環境**的動作，只會寫進上下文，供之後的步驟使用。執行流程如下：

```
Thought → Action → Observation → Thought → Action → Observation → … → Finish
```

- **Thought**：分解目標、追蹤進度、從觀察中擷取重點、調整計畫。
- **Action**：呼叫環境提供的工具（HotpotQA 中是 `search[entity]`、`lookup[string]`、`finish[answer]`）。
- **Observation**：環境回傳的結果，附加到上下文。

模型靠 prompt 中**人工撰寫的 few-shot 範例軌跡**（論文附錄 C）學會這個格式。Act baseline 則是把同樣的範例刪掉所有 Thought 行。

論文的主要結果（PaLM-540B）：

任務	Act	ReAct
HotpotQA（EM）	25.7	27.4
FEVER（Acc）	58.9	60.9
ALFWorld（成功率，best of 6）	45	71
WebShop（成功率）	30.1	40.0

1.4 作者面臨的挑戰、解法與取捨

挑戰	解法 / 取捨
CoT 的幻覺	透過行動查證外部知識。論文 Table 2：ReAct 失敗案例中幻覺占 0%，CoT 占 56%
結構限制降低推理彈性	Thought–Action 必須交錯，使 ReAct 的推理錯誤率（47%）高於 CoT（16%），也較常陷入重複同樣的 Thought 與 Action。作者明確稱之為「factuality 與 flexibility 的取捨」
搜尋結果無用時難以恢復	失敗案例中 23% 來自搜尋結果無用。解法是 **ReAct 與 CoT-SC 互相 fallback**（ReAct 步數用完改用 CoT-SC；CoT-SC 多數決不夠穩定改用 ReAct），兩個任務的最佳成績都來自這種組合

挑戰	解法 / 取捨
小模型學不會	PaLM-8B/62B 用 prompting 時 ReAct 是四種方法中**最差**的（「difficulty to learn both reasoning and acting from in-context examples」）；改用 3,000 條軌跡 finetune 後才變成最好
人工標註軌跡的成本	只用 1–6 條範例 prompting；finetuning 改用模型自己產生且答案正確的軌跡（bootstrapping）

1.5 本報告的研究問題

論文的任務是問答與決策。我想知道：**同樣的消融實驗搬到「單元測試生成」上，結論還成立嗎？**

測試生成很適合檢驗 ReAct，原因有兩個：

- 它天生需要「讀規格 → 推理期望值 → 寫測試 → 判斷失敗原因」，和 ReAct 的 Thought–Action–Observation 結構對應。
- 它有**客觀的量尺**：可以把測試跑在正確版和 bug 版上，判斷測試到底有沒有抓到 bug，不必依賴模型的自我報告。

---

2. Implementation

2.1 實驗環境與設定

項目	設定
模型	`gpt-oss:20b`（Ollama ID 17052f91a42e，13 GB）
推論框架	Ollama 0.30.7，原生 API `/api/chat`
推理強度	`think = "low"`（延伸實驗另有 `high`）
上下文長度	`num_ctx = 16384`（high 條件為 32,768）
內部思考是否送回歷史	主實驗：否；延伸實驗：另設保留的條件
最大步數	25（不告訴模型，`+limit` 條件除外）
溫度	Ollama 預設值
其他	Python 3.12、pytest 9.1.1、coverage 7.16.2

開發分成 8 個步驟，每一步都有獨立的程式和 git commit：

步驟	檔案	內容
1	`step1_call_llm.py`	透過 HTTP 呼叫 LLM，確認它沒有記憶、輸出有隨機性
2	`step2_tools.py`	工具（環境）
3	`step3_act_agent.py`	Act agent
4	`step4_react_agent.py`	加入 Thought（ReAct）
5	`step5_fewshot_agent.py`	加入 few-shot 範例
6	`step6_agent.py`	最終版：結束條件、重試、所有條件參數化
7	`step7_evaluate.py`	獨立評分
8	`step8_experiment.py`	批次執行、彙整、排列檢定

2.2 被測程式與植入的 bug

subject/cart.py 是我自己寫的購物車計價模組（約 160 行），規格寫在每個函式的 docstring 中。我先寫正確版，再植入 4 個 bug。agent 只拿得到含 bug 的版本，也不知道 bug 在哪裡。

ID	函式	內容	類型
B1	bulk_discount_percent	剛好 50 件應打 10% 折扣，程式給 5%（>= 寫成 >）	邊界
B2	parse_coupon	規格不分大小寫，程式分大小寫（少了 .upper()）	規格功能
B3	Cart.remove_item	數量歸零後商品未被移除（<= 寫成 <）	狀態
B4	Cart.total	運費應以套用優惠碼「後」的金額計算，程式用「前」的金額	多跳

B4 需要串連 total、parse_coupon、shipping_fee 三份規格，並設計一個「優惠前 ≥ 1000、優惠後 < 1000」的輸入才能發現，對應 HotpotQA 的多跳推理。

以 B1 為例，規格與植入 bug 的實作：

```
def bulk_discount_percent(quantity):
    """...
        quantity >= 100  -> 15
        quantity >= 50   -> 10
        quantity >= 10   -> 5
    """
    if quantity >= 100:
        return 15
    if quantity > 50:        # B1：規格是 >= 50
        return 10
```

2.3 工具：分開「規格」與「實作」以觀察 assertion echo

工具	回傳內容	對應論文
list_functions	函式與參數清單	類似 search
read_spec	只有 docstring（規格）	類似 search
read_source	實作程式碼	類似 lookup
write_tests	寫入 test_cart.py	—
run_tests	執行 pytest，回傳結果	—
finish	結束並回報	finish[answer]

刻意把 read_spec 和 read_source 分開，是為了觀察**期望值的來源**：

期望值來源	對 parse_coupon("save10") 寫出的測試	結果
規格	== 10	測試失敗，**抓到 bug**
程式現有行為	raises ValueError	測試通過，**bug 被藏起來（assertion echo）**

2.4 Agent 架構

agent 是一個最多 25 步的迴圈：把對話紀錄（messages）交給模型，模型決定呼叫哪個工具，程式執行後把結果（Observation）接回對話紀錄，直到模型呼叫 `finish` 或步數用完。agent 最後只留下 `test_cart.py`，交給 2.7 節的評分程式打分；評分用的 6 個 cart 版本 agent 都看不到。

設計決策一：改用原生工具呼叫

我一開始照論文用純文字格式，讓模型輸出 `Action: read_spec[parse_coupon]` 再由程式解析，但 gpt-oss 一直回傳空白或讓伺服器 HTTP 500。gpt-oss 訓練時就內建了原生工具呼叫，文字格式反而和它衝突，所以我改用 Ollama 的 `tools` 參數：事先用 JSON Schema 描述每個工具，模型直接回傳 `{"name": "read_spec", "arguments": {"name": "parse_coupon"}}` 這樣的結構。

設計決策二：把 Thought 做成 thought 參數

原生工具呼叫只回傳結構化的參數，沒有地方寫 `Thought:` 那一行。我的做法是在 ReAct 模式下，給每個工具多加一個必填參數 `thought`：

```
Act:   {"name": "read_spec", "arguments": {"name": "parse_coupon"}}
ReAct: {"name": "read_spec", "arguments": {"thought": "I need to know the coupon rules first.",
                                           "name": "parse_coupon"}}
```

程式執行工具前先把 `thought` 拿掉，所以工具完全不受影響；但整個呼叫（包含 `thought`）會原封不動存進對話紀錄，下一步看得到。這正好對應論文中 Thought 的兩個特性：不改變環境、留在上下文中（$\hat{A} = A \cup L$）。對應的程式碼：

```python
# 建立工具清單時：ReAct 模式下，每個工具多一個必填欄位 thought
params["properties"] = {"thought": {"type": "string", "description": THOUGHT_DESC},
                        **params["properties"]}
params["required"] = ["thought"] + params["required"]

# 執行工具時：先把 thought 拿出來，剩下的參數才交給工具
args = dict(call["function"]["arguments"])
thought = args.pop("thought", None)
observation = TOOLS[name](**args)
```

Act 和 ReAct 只差兩處，其他（模型、工具、題目、步數上限）完全相同：

1. ReAct 的每個工具多一個 `thought` 參數；
2. ReAct 的 system prompt 多一句：「Every tool has a thought argument. Before each action, use it to reason about the current situation.」（取自論文 HotpotQA prompt 的說法）

我刻意不告訴模型 Thought「該想什麼」。如果寫了「請先想想邊界值」之類的提示，ReAct 變好時就分不清是「寫 Thought」的功勞，還是「提示內容」的功勞。

容易混淆的地方：Thought 不等於模型的內部思考

gpt-oss 是**推理模型**，每次回答前都會自動先想一段，稱為**內部思考（thinking）**。這是模型內建的行為，無法完全關閉。所以這個實驗中其實有兩種推理：

	內部思考（thinking）	Thought
怎麼產生	模型每一步自動產生	我要求模型填在 `thought` 參數裡
Act 有沒有	有	沒有
ReAct 有沒有	有	有
下一步看得到嗎	主實驗中**看不到**（程式沒有把它放回對話紀錄）	**看得到**

因此，主實驗中 Act 和 ReAct 真正的差別是：這一步的推理，下一步還看不看得到。（2.10 節的延伸實驗會改成把內部思考也留下來。）

上表的兩個「看不看得到」都有實際驗證：

- **Thought 有留下來**：手動組一段對話紀錄，把暗號 PURPLE-ELEPHANT-42 只藏在上一步的 thought 參數裡，再問模型「你上一步的 thought 寫了什麼暗號？」模型答得出來；對照組（thought 裡沒有暗號）則答不出來，排除了瞎猜的可能。
- **內部思考的去留**：用 Ollama 回報的「輸入 token 數」直接測量。在對話紀錄中放入一段約 300 token 的內部思考，若輸入 token 數因此多了約 300，就代表它真的被送進模型。結果顯示：Ollama 只會保留「最後一則 user 訊息之後」的內部思考，在那之前的會被丟掉。agent 的對話紀錄只有開頭一則 user 訊息，所以只要程式把內部思考放回對話紀錄，它就會被保留（這是 2.10 節 keep 條件的做法）；主實驗則沒有放回。

2.5 few-shot 範例

論文中 ReAct 的 Thought 之所以寫得好，很大一部分來自 prompt 裡人工撰寫的示範軌跡。我也寫了一份，但如果直接示範怎麼測 cart，等於把答案告訴模型。所以改用**另一個模組 thermo**（溫度轉換），示範完整的一輪：讀規格 → 寫測試 → 跑測試 → 判斷失敗原因 → 結束。範例裡的每個 Observation 都是實際執行工具得到的輸出，不是我編的。

範例中最關鍵的一段，是示範測試失敗時該怎麼判斷：是測試寫錯了，還是程式違反規格？

```
Thought: Two failures. (1) c_to_f(36.6): 36.6 * 9 / 5 + 32 = 97.88, but the spec says
round to 1 decimal, so the correct value is 97.9. MY expected value was wrong, so I fix
the test. (2) classify(25) returned 'mild', but the spec says celsius >= 25 is 'hot'.
That expected value comes from the spec, so the MODULE is wrong; I keep this test unchanged.
```

Act 版拿到的是同一份範例，只是刪掉了所有 `Thought:` 行（論文的 Act baseline 也是這樣做）。

2.6 實驗條件

實驗有三個開關：

- **Thought**：是否要求模型寫 Thought（也就是 Act 還是 ReAct）；
- **few-shot 範例**：是否附上 2.5 節的示範；
- **告訴步數上限**：是否在 prompt 中告訴模型「你最多只有 25 步」。這是額外的假設：知道有上限，agent 會不會規劃得比較好？

組合出以下 5 個條件：

條件	Thought	few-shot 範例	告訴步數上限	對應論文
`act`	—	—	—	Act baseline
`react`	✓	—	—	ReAct（$\hat{A} = A \cup L$）
`act+fewshot`	—	✓（刪除 Thought 行）	—	附錄 C 的 Act prompt
`react+fewshot`	✓	✓	—	附錄 C 的 ReAct prompt
`react+fewshot+limit`	✓	✓	✓	額外假設

執行前先定好要回答的四個問題：

1. 沒有範例時，加 Thought 有沒有幫助？（act vs react）
2. 有範例時，加 Thought 有沒有幫助？（act+fewshot vs react+fewshot）

3. 加範例有沒有幫助？（沒有範例的 200 次 vs 有範例的 200 次。前者是 act + react，後者是 act+fewshot + react+fewshot）
4. 告訴步數上限有沒有幫助？（react+fewshot vs react+fewshot+limit）

每個條件跑 100 次，因為同一個條件可能這次抓到 0 個 bug、下次抓到 3 個（見 2.9 節 (c)），只跑幾次看到的差異很可能只是運氣。五個條件輪流執行，讓「某段時間電腦比較忙」之類的因素平均分攤。

差異是否顯著用排列檢定判斷：如果兩組其實沒有差別，把 200 個結果打散、隨機重新分成兩組，應該也常出現同樣大的差距。重複隨機分組 10,000 次，「隨機分組就出現這麼大差距」的比例就是 p 值。本實驗使用雙尾檢定、固定 seed，結果可重現。

2.7 獨立量尺（評分方式）

我不採信 agent 自己回報「找到幾個 bug」，而是把它寫的測試拿到不同版本的 cart 上跑，看每個測試在哪些版本通過、哪些失敗。

我準備了 6 個版本的 cart（下文用 bug 編號稱呼只含該 bug 的版本，例如「B1 版」就是只含 B1、其他地方都正確的 cart）：

版本	內容	用途
fixed	沒有 bug 的正確版	檢查測試本身有沒有寫錯
buggy	4 個 bug 全部都在（就是 agent 拿到的版本）	檢查測試能不能抓到 bug
B1 版～B4 版	各只含一個 bug，其他地方都正確	判斷測試抓到的是哪一個 bug

測試的分類

每個測試依兩個問題分類：在正確版 fixed 上通過嗎（期望值對不對）？在含 bug 的版本上結果和正確版不同嗎（對 bug 有沒有反應）？

分類	在 fixed 上	在含 bug 的版本上	意義
bug-revealing	通過	至少一個版本失敗	期望值正確，而且抓到 bug
passing	通過	全部通過	期望值正確，但沒碰到任何 bug
assertion-echo	失敗	至少一個版本通過	期望值等於 bug 的錯誤結果，bug 反而被藏起來
invalid	失敗	全部失敗	期望值在哪個版本都不對（誤報）

實際例子

以 B1 為例：規格說剛好 50 件應打 10% 折扣，但 B1 的程式給 5%。正確版、B2、B3、B4 的 bulk_discount_percent(50) 都回傳 10，只有 buggy 和 B1 回傳 5。下表是 4 個測試在 6 個版本上的結果（✓ 通過、✗ 失敗）：

測試	fixed	buggy	B1 版	B2 版	B3 版	B4 版	分類
bulk_discount_percent(50) == 10	✓	✗	✗	✓	✓	✓	bug-revealing
bulk_discount_percent(100) == 15	✓	✓	✓	✓	✓	✓	passing

測試	fixed	buggy	B1 版	B2 版	B3 版	B4 版	分類
`bulk_discount_percent(50) == 5`	✗	✓	✓	✗	✗	✗	assertion-echo
`bulk_discount_percent(50) == 7`	✗	✗	✗	✗	✗	✗	invalid

其中 `== 5` 照抄了有 bug 的程式，在 agent 拿到的 buggy 上會通過，agent 就以為程式沒問題；`== 7` 在哪個版本都失敗，開發者會以為程式有 bug，其實是測試寫錯了。

報告中的指標怎麼算

- **抓到的 bug（/4）**：B1–B4 中，有幾個至少被一個 bug-revealing 測試抓到。例如一份測試檔有 3 個測試抓到 B1、1 個測試抓到 B3，就算抓到 2 個 bug。
- **invalid（誤報）**：一份測試檔中有幾個 invalid 測試。
- **assertion-echo**：一份測試檔中有幾個 echo 測試。
- **line / branch coverage**：用 coverage.py 量測測試在正確版上執行到多少比例的程式行和分支，代表測試「跑過」多少程式碼。

還有一種少見的情況：某個測試在每個單一 bug 版本上都通過，只有在 4 個 bug 同時存在的 buggy 上才失敗。這種測試仍算 bug-revealing（標為 `combined`），但無法歸給 B1–B4 中的任何一個，所以不計入「抓到的 bug」。

評分程式本身也要先驗證

評分程式一旦寫錯，整個實驗的結論都會跟著錯，所以我先用答案已知的測試檢查它：

- **先依規格的參考測試**：應該要抓到全部 bug、沒有錯誤。結果是 4/4 個 bug、0 個 echo、0 個誤報，line / branch coverage 都是 100%。
- **8 個事先知道正確分類的測試**（例如上表那種，含 B1–B4 各一個 echo）：全部被分類正確。

2.8 前導觀察（單次執行的質性發現）

正式跑 500 次之前，我先跑了幾次並逐步閱讀 agent 的軌跡。這些是個別案例，不是統計結論，但能解釋後面的數字，也影響了後續的設計。

- **Thought 只是在描述動作。** ReAct 的 Thought 幾乎都像 `Spec of apply_discount.`、`Spec of parse_coupon.` 這樣，平均只有約 25–35 個字元，真正的推理在內部思考裡。下表是同一步的兩者對照：

內部思考（下一步看不到）	Thought（下一步看得到）
`Discrepancies: bulk_discount_percent 50 should be 10 but returns 5 => bug. parse_coupon case-insensitive fails...`	`Final summary of mismatches.`

  模型乖乖填了 Thought 欄位，但真正的思考放在內部思考，而主實驗中內部思考下一步就看不到了。這是 2.10 節延伸實驗的動機。

- **Thought 說的和實際做的不一樣。**某次 agent 重寫了測試檔，但兩個版本之間只差註解，斷言完全沒變，Thought 卻寫著「Updated tests to match spec: …」。論文宣稱 Thought 能提供「可解釋性」，但如果 Thought 和行動對不上，這個好處就要打折扣。
- **沒讀規格就猜介面。** `Cart.items()` 回傳什麼格式寫在它的規格裡，但所有前導執行都沒有去讀，而是各自猜了不同的格式，於是寫出錯誤的測試（誤報）。

- **assertion echo 的實際案例**（react）。agent 連續呼叫 9 次 read_source（讀程式碼）才開始寫測試，測試中的註解洩漏了期望值是從哪裡來的：

  ```
  cart.add_item("gadget", 15, quantity=50)  # 750 base, discount 5% (since <100 and >50 triggers 5%)
  ```

  規格是「>= 50 打 10%」，註解卻寫「>50」，可見期望值是抄自有 bug 的程式碼。有 Thought 也沒有阻止它跳過規格。
- **憑直覺猜也可能產生 echo**（react）。agent 沒有讀程式碼，卻寫出 `total == 800.0`（依規格應該是 830），剛好等於 B4 的錯誤結果。當 bug 本身就是常見的錯誤直覺時，憑直覺猜也會猜出同樣的錯誤答案。
- **為了讓測試通過而把測試改壞**（act）。第一版測試有 4 個失敗，很可能就是真正的 bug，但 agent 把測試改寫了 5 次，最後只剩 1 個測試函式，一個 bug 都沒抓到（0/4）。agent 的目標變成「讓測試通過」，而不是「找出 bug」。

2.9 主實驗結果（每條件 n = 100，共 500 次）

500 次執行中，494 次正常呼叫 `finish` 結束，2 次因連續重複同一個工具呼叫（`loop`）被中止，4 次用完 25 步（`step_limit`），沒有任何一次因模型錯誤而中斷。總耗時 190 分鐘。

四個問題的答案：

問題	答案	依據
1. 沒有範例時，加 Thought 有幫助嗎？	沒有，反而有變差的跡象	抓到的 bug 1.76 → 1.45；讀的規格 4.6 → 3.1 份（顯著變少）
2. 有範例時，加 Thought 有幫助嗎？	沒有顯著差異	抓到的 bug 2.37 → 2.51（不顯著），輸入 token 卻顯著增加約 22%
3. 加範例有幫助嗎？	有，而且是所有因素中最大的	抓到的 bug +0.83、誤報 −55%，都顯著
4. 告訴步數上限有幫助嗎？	沒有	各項指標都沒有顯著差異

(a) 測試品質

「抓到的 bug」是每次執行的平均（滿分 4）；B1–B4 列是 100 次中抓到該 bug 的次數；invalid 是每次執行平均的誤報測試數。

指標	act	react	act+fewshot	react+fewshot	+limit
抓到的 bug（/4）	1.76	**1.45**	2.37	**2.51**	2.43
B1 邊界	72	61	87	88	88
B2 大小寫	77	70	93	89	92
B3 移除商品	23	11	53	67	61
B4 多跳	4	3	4	7	2
assertion-echo 測試	0.1	0.1	0.1	0.1	0.0
invalid（誤報）	2.8	3.2	1.4	1.3	1.0
line coverage %	82.7	82.7	91.3	93.8	93.6
branch coverage %	74.4	73.6	86.2	90.2	90.2

左邊兩欄（沒有範例）和右邊三欄（有範例）差距明顯，加不加 Thought 則差距很小。B4（多跳 bug）在所有條件都只有 2–7%。

(b) agent 行為

cart 共有 9 份規格可讀。`Cart.items` 的規格特別重要，不讀就會猜錯回傳格式而產生誤報（見 2.8 節）。

指標	act	react	act+fewshot	react+fewshot	+limit
讀了幾份規格（/9）	4.6	**3.1**	7.1	7.7	8.3
讀了 `Cart.items`（次 / 100）	27	**1**	63	71	86
步數	10.9	10.3	14.4	15.9	16.0
Thought 平均字元數	—	24.8	—	27.3	25.1

react 尤其明顯：100 次中只有 1 次讀了 `Cart.items` 的規格。加了 Thought 之後，agent 反而更常跳過讀規格、直接寫測試。

(c) 結果的變異：抓到的 bug 數分布（100 次中各有幾次）

條件	0 個	1 個	2 個	3 個	4 個
`act`	20	9	48	21	2
`react`	25	10	60	5	0
`act+fewshot`	4	7	39	48	2
`react+fewshot`	8	5	21	60	6
`react+fewshot+limit`	7	3	31	58	1

每個條件都同時出現過「一個都沒抓到」和「抓到 3 個以上」，同一個設定只跑一次的結果幾乎沒有參考價值。每次寫出的測試數量差異也很大（例如 act 從 1 個到 49 個都有）。

(d) 成本（每次執行的平均）

指標	act	react	act+fewshot	react+fewshot	+limit
輸入 token	18,169	22,256	46,732	57,001	57,308
輸出 token	2,087	2,265	2,680	2,733	2,615
耗時（秒）	17.1	19.7	22.3	24.7	25.1

模型在本機 GPU 上執行，沒有 API 費用，所以成本以 token 和時間表示。「輸入 token」是每一步送進模型的對話紀錄長度的加總，對話越長、步數越多就越高。範例本身很長，又讓 agent 走更多步，所以加範例後輸入 token 增加最多；加 Thought 則一律讓成本變高。

(e) 統計檢定

每一格是「箭頭右邊減去左邊」的差，括號內是 p 值。共 4 個比較 × 4 個指標 = 16 次檢定，為了避免多重比較「剛好」出現顯著，用 Bonferroni 校正把門檻改成 0.05 ÷ 16 ≈ 0.003，粗體為顯著。

比較	抓到的 bug	invalid	讀了幾份規格	輸入 token
Thought（無範例）act → react	−0.31（p = 0.035）	+0.42（p = 0.35）	**−1.54（p < 0.001）**	+4,087（p = 0.041）
Thought（有範例）act+fewshot → react+fewshot	+0.14（p = 0.32）	−0.12（p = 0.60）	+0.54（p = 0.062）	**+10,269（p < 0.001）**
範例（n = 200 對 200）	**+0.83（p < 0.001）**	**−1.66（p < 0.001）**	**+3.51（p < 0.001）**	**+31,654（p < 0.001）**
告訴上限 react+fewshot → +limit	−0.08（p = 0.60）	−0.27（p = 0.17）	+0.66（p = 0.011）	+307（p = 0.89）

無範例時加 Thought，唯一顯著的是讀的規格變少；抓到的 bug 少 0.31 個（p = 0.035）沒有通過門檻，只能算有變差的跡象。有範例時加 Thought，只有成本顯著增加。範例則四項指標全部顯著，代價是輸入 token 多了 3 萬多。

2.10 延伸實驗：推理放在哪裡才有用？

主實驗顯示 Thought 沒用，但這是「推理」沒用，還是「Thought 這種形式」沒用？主實驗留下兩個疑問：

- **Q1：推理留不住，是不是問題所在？** 2.9 節 (b) 顯示 Thought 平均只有約 25 個字元，真正的推理在內部思考裡，而主實驗把內部思考丟掉了。如果改成把內部思考留在對話紀錄中，讓下一步看得到，會怎麼樣？
- **Q2：B4 抓不到，是因為想得不夠深嗎？**有範例時，約 70% 的執行都讀了 `Cart.total` 的規格，B4 卻只抓到 2–7%。也就是說 agent 讀到了相關規格，但沒有推出「需要設計一個優惠前 ⩾ 1000、優惠後 < 1000 的輸入」。如果讓模型每一步想得更久呢？

為此新增 4 個條件：

- **keep**：把內部思考放回對話紀錄，讓下一步看得到。act+keep 代表「有推理也留得住，但不是被要求寫的 Thought」。
- **high**：把推理強度從 low 調到 high，讓模型每一步想得更久；上下文長度同時加大到 32,768（原因見本節最後）。

這些問題是看過主實驗結果後才提出的，屬於探索性實驗（但比較項目在執行前就先定好），可信度比主實驗略低。

條件	n	抓到的 bug	B4	invalid	讀規格	輸入 token	耗時（秒）
act（主實驗）	100	1.76	4%	2.8	4.6	18,169	17.1
react（主實驗）	100	1.45	3%	3.2	3.1	22,256	19.7
act+keep	100	**2.11**	10%	1.8	**6.1**	24,094	24.2
react+keep	100	2.02	11%	1.3	4.6	40,315	30.6
act+fewshot+high	50	**3.18**	**66%**	**0.4**	7.7	91,490	**222.7**
react+fewshot+high	50	2.76	**50%**	0.8	7.0	104,570	**206.5**

統計檢定共 27 次，Bonferroni 門檻為 0.05 ÷ 27 ≈ 0.0019。主要結果：

- **Q1：是。**比較 react → act+keep：兩者都把推理留在對話紀錄中，成本也相近，差別只在留下的是被要求寫的 Thought 還是模型自己的內部思考。結果 act+keep 多抓 0.66 個 bug、誤報少 1.39 個、多讀 3.04 份規格，三項都顯著。模型自然產生的推理，比被要求寫的 Thought 有用得多。
- **已經保留內部思考時再加 Thought**（act+keep → react+keep）：抓到的 bug 沒有差異，但讀的規格顯著變少（6.1 → 4.6），輸入 token 多了 67%。
- **Q2：是。**比較 act+fewshot → act+fewshot+high：B4 抓到率從 4% 提高到 66%，抓到的 bug +0.81、誤報 −0.95，都顯著。代價是耗時約 10 倍、輸出 token 約 11 倍。high 條件的結果也很兩極：60% 的執行抓到全部 4 個 bug，但 12% 一個都沒抓到，這些執行大多一直在思考、讀檔，直到 25 步用完都沒寫出測試。

high 條件把上下文加大到 32,768，是因為試跑時有一步的輸入加輸出剛好等於上限 16,384，內部思考寫了 5 萬多字元，模型還沒想完空間就用光了，來不及呼叫工具。我原本的檢查寫成「超過 16,384」，剛好等於的情況就被漏掉了。另外 high 同時改了推理強度和上下文長度，兩者的效果無法完全分開。

2.11 Assertion echo 檢查

作業要求檢查測試的期望值是否來自規格，而不是照抄程式現在的行為（assertion echo，定義見 2.3 節）。結果：

echo 其實很少見。依 2.7 節的評分方式，平均每次執行只有約 0.1 個 echo 測試；每個條件的 100 次中，含有 echo 的執行只有 4–9 次。真正的問題反而是誤報和沒讀規格就猜，也就是期望值寫錯（invalid），以及沒讀 `Cart.items` 的規格就猜回傳格式。

echo 少，可能是因為 system prompt 規定「期望值必須來自規格」。前導實驗中，Act 的內部思考曾出現 `We need adjust tests to match implementation. But we should just report mismatches.`，模型的第一反應是修改測試去配合程式，是這條規則把它拉回來。不過這點我沒有做對照實驗驗證。

從 2.8 節的兩個案例來看，echo 有兩種來源：一是讀程式碼、照抄現有行為；二是 bug 剛好等於常見的錯誤直覺。後者即使完全不讀程式碼也無法避免。

2.12 工程挑戰與可靠性

問題	原因	解法
模型回應空白、HTTP 500	gpt-oss 內建了自己的工具呼叫格式，和論文的文字格式衝突	改用 Ollama 原生 `tools` 參數（見 2.4 節）
第一批 25 次中有 9 次中斷（36%）	錯誤訊息為 `HTTP 500: error parsing tool call`，9 次中有 8 次發生在 `write_tests`：把一長段程式碼塞進 JSON 字串時，跳脫字元出錯	同一個請求最多重試 3 次，仍失敗就告訴模型格式錯誤。加入後 500 次中 0 次中斷
中斷的執行會被算成「抓到 0 個 bug」，拉低平均	例如 react 5 次中有 3 次中斷，平均只抓到 0.4 個	這批結果保留作為可靠性問題的證據，加入重試後全部重跑
工具回傳的結果夾雜 coverage 警告	上層資料夾的 pytest 設定含有 `--cov`	執行 pytest 時加上 `-o addopts=`
同時跑兩批實驗會互相干擾	所有 agent 共用同一個 `workspace/test_cart.py`	一次只跑一批，用 tmux 管理
長時間實驗可能中途停止	斷線、伺服器重開機	`--runs` 改成「每個條件總共跑 N 次」，中斷後重跑同一個指令就能接著跑

其中最值得注意的是：用原生工具呼叫傳遞一長段程式碼時特別容易出錯，實作測試生成 agent 時要特別小心。

2.13 Threats to validity

面向	說明
能否推廣	只測了一個被測程式（約 160 行）和一個模型（gpt-oss:20b），換成其他程式或模型，結論可能不同
指標太粗	只有 4 個 bug：B1、B2 太簡單（70–90% 都抓得到），B4 太難（只有 2–7%），條件之間的差異主要靠 B3 區分
範例的效果混了兩件事	範例的 Thought 裡有具體的測試策略（測邊界值、讀完所有規格、不要猜），範例中的 bug 也和 B1 同類型。Act 版刪掉 Thought 行時，這些策略也一起被刪了。所以「範例的效果」其實混合了「模仿範例」和「策略提示」，無法分開

面向	說明
Act 也有推理	gpt-oss 的內部思考無法完全關閉，所以 Act 並不是「完全沒有推理」，只是推理留不到下一步
規格本身有瑕疵	`Cart` 類別的說明寫出了內部儲存格式（`name -> (unit_price, quantity)`），可能誘導 agent 猜錯 `items()` 的回傳格式。我刻意保留，因為真實世界的規格也常這樣
統計	Bonferroni 校正是看到結果之後才決定採用的；延伸實驗的問題也是看過主實驗結果才提出的
成本被低估	因格式錯誤而重試的模型呼叫沒有計入 token。react 類條件的重試比較多，所以 Thought 的實際成本比表中更高
溫度	使用 Ollama 預設值，沒有明確設定

---

3. Reflections

3.1 結論摘要

有幫助的	沒有幫助的
few-shot 範例（模仿「讀完所有規格」等動作，混合策略提示）	要求寫 thought 參數（4 組比較中沒有一組顯著抓到更多 bug）
保留模型自己的內部思考	告訴模型步數上限
提高推理強度（尤其是多跳 bug）	

推理有用，但要是模型自己的推理，而不是被要求填寫的欄位。ReAct 的核心精神（把推理寫回上下文）在 react → act+keep 的比較中得到支持；沒有重現的，是「另外要求寫一個 Thought」這種形式。

3.2 ReAct 論文的優點

- **概念簡單、可遷移**：Â = A ∪ L 只是在動作空間加一個「說話」的動作，幾行程式就能實作；我能把它搬到完全不同的任務上做消融實驗，顯示它很容易移植。
- **消融實驗設計乾淨**：Act baseline 就是「同一條範例刪掉 Thought 行」，只改一個變因。本報告沿用這個方法，才能做公平比較。
- **作者誠實寫出限制**：論文明確指出結構限制導致推理錯誤增加、小模型 prompting 時 ReAct 最差。這和本報告的觀察方向一致。

3.3 ReAct 論文的缺點

- **Thought 的品質高度依賴範例**：論文的 Thought 很漂亮，是因為範例很漂亮。本實驗中，模型模仿了範例的**動作**，卻沒有模仿**推理**（範例中的 `Still unread: …`、`I must not guess` 從未出現）。
- **可解釋性被高估**：Thought 可能與實際行動不符（2.8 節）。論文把 Thought 當作可以信任的解釋，但它其實只是模型的另一個輸出。
- **時代限制**：論文使用非推理模型（PaLM），Thought 是唯一的推理管道。現代推理模型（gpt-oss、o 系列等）已有內部思考，Thought 欄位變成重複，甚至干擾（本實驗中讓 agent 少讀規格）。
- **成本沒有充分討論**：Thought 一律增加 token（本實驗 +22%），而且 few-shot 範例每一步都要重送（輸入 token 約 2.6 倍）。

3.4 改進建議

1. **用推理強度取代 Thought 欄位**：對推理模型，保留內部思考或提高推理強度，比要求寫 Thought 更有效、更便宜。
2. **把策略寫成規則**：把範例中的測試策略（邊界值分析、讀完所有規格、不要猜介面）直接寫進 system prompt，不給範例，拆開「模仿」與「策略提示」的效果。
3. **加入一致性檢查工具**：例如「列出尚未讀過規格的函式」，彌補 agent 沒有進度追蹤的弱點。
4. **用 mutation score 取代人工植入 bug**：4 個 bug 的解析度太低；mutation testing 工具（例如 mutmut）可以自動產生大量 mutant，作為更細緻的量尺。
5. **避免長程式碼放進 JSON 參數**：例如改成多次小量寫入，降低工具呼叫的格式錯誤。

3.5 哪些地方仍需要人介入

- **寫 oracle**：這次實驗能客觀評分，是因為我手上有正確版和植入 bug 的清單。真實專案沒有這些，期望值對不對最終還是要人依規格判斷。
- **審查失敗的測試**：主實驗中誤報（invalid，平均 1.0–3.2 個）遠多於 echo（約 0.1 個）。agent 回報「測試失敗」時，要有人分辨是程式有 bug 還是測試寫錯。
- **審查 agent 的說法**：2.8 節中 Thought 寫「Updated tests to match spec」，實際上斷言完全沒變；agent 的自我報告不能直接採信。
- **補上難的測試案例**：多跳的 B4 在主實驗中只有 2–7% 被抓到。需要串連多份規格的邊界條件，目前還是要靠人設計。

3.6 對於論文的看法

雖然這篇是 2022 年的論文，放到現在已經算有點過時了，但我覺得這篇論文的理念還是很值得一看，畢竟那時的 LLM 寫出來的東西經常出錯，閱讀完就會慢慢理解什麼叫做多一層思考。後來的推理模型（像這次用的 gpt-oss）也內建了類似的思考步驟。透過一步一步模仿論文製作 agent，也讓我更具體地理解平常在用的 Claude Code 是如何運作的。

3.7 與其他工具的比較及未來是否採用

老實說，實際要寫測試的話，Claude Code 還是強很多。它背後的模型比我本地跑的 gpt-oss:20b 大很多，工具也比較完整，我自己寫的 ReAct agent 比不上。不過自己寫的 agent 有一個好處：每個變因都能自己控制，Thought、few-shot、推理強度都可以一個一個拿掉來比較，這是用現成工具做不到的。因此研究方面，我蠻想繼續往這個方向找題目。

---

4. AI 使用聲明

本報告與程式的準備過程中使用了以下 AI 工具：

工具	用途
Claude Code（Anthropic，Claude Opus）	審查評分程式 `step7_evaluate.py`，指出漏判「多個 bug 同時存在」的情況，因而補上 `combined` 判斷（已確認不影響主實驗結果）；協助整理實驗紀錄，並依實驗紀錄撰寫本報告初稿；之後協助精簡改寫報告、依我的意見撰寫部分段落，並轉成 PDF
gpt-oss:20b（OpenAI，經 Ollama 本機執行）	實驗對象：作為產生測試的 agent 模型，不用於撰寫報告

實驗設計、被測程式、植入的 bug、few-shot 範例、評分標準與所有數據，都來自我的實驗；報告中的每個數字都可以在 runs/ 中的紀錄找到來源。

參考資料

1. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. ICLR 2023. arXiv:2210.03629.
2. Wei, J. et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.
3. Ollama API 文件：`/api/chat`、tools 參數。
4. 本專案實驗紀錄：`README.md`（主實驗）、`Extra.md`（延伸實驗）。